

LLM and Agentic RAG Infrastructure Summary

Complete component inventory for integrating an LLM with the existing Neon/pgvector paper database, adding bounded agentic retrieval, validating scientific answers, and exposing the workflow through a simple Streamlit application. Status reflects the local worktree as of 31 August 2026.

1. Target system

The finished application will accept a fast-charging research question, embed it with the same model used for the database, retrieve evidence from Neon, allow a language model to perform a limited number of follow-up retrieval operations, and return structured protocol suggestions with citations, limitations, missing conditions, and safety warnings.

User prompt	Query embedding	Neon retrieval	Bounded LLM agent	Validated answer
-------------	-----------------	----------------	-------------------	------------------

Current position

- The application shell, contracts, safeguards, agent loop, CLI, Streamlit UI, and offline tests have been implemented.
- The teammate's two-stage vector retrieval and cross-encoder reranking pipeline has been pulled and connected to the live application adapter.
- The adapter now implements semantic search, neighboring-chunk retrieval, paper metadata retrieval, pre-retrieval section filters, and paper-specific searches.
- A local Git-ignored .env file is loaded automatically; credential values and the Parley model ID are not yet configured.
- No live Parley request or live end-to-end Neon retrieval has been tested.
- All new application files remain uncommitted and should be reviewed before merging.

2. Core application scripts

Component / script	Primary function	Progress	Remaining work
app/config.py	Loads the repository-root .env file and reads Neon, Parley, model, timeout, evidence-size, output-token, search, and agent-round settings. Existing process variables retain priority.	Built	Enter DATABASE_URL, LLM_API_KEY, and LLM_MODEL locally; later add provider-specific credit/budget settings.
app/schemas.py	Defines provider-neutral objects for search filters, evidence chunks, tool calls, token usage, model replies, validation results, and final agent results.	Built	Extend fields if the revised retrieval schema exposes section paths, DOI, authors, or richer provenance.
app/interfaces.py	Defines stable RetrievalBackend and LLMBackend contracts. This keeps the agent and UI isolated from retrieval and provider internals.	Built	Extend only if evaluation reveals a need for additional retrieval operations.
app/retrieval_adapter.py	Reuses the shared GTE embedder, pgvector candidate search, and GTE cross-encoder reranker; implements search_chunks, fetch_neighbors, and get_paper_metadata; maps ranking and provenance into EvidenceChunk.	Built	Run live Neon checks; validate schema assumptions, model downloads, latency, zero-result behavior, and model reuse under Streamlit.
app/llm_client.py	Calls Parley or another OpenAI-compatible chat endpoint, submits tool definitions, parses tool calls, handles timeouts/429 errors, and records token usage.	Built	Test against Parley; confirm base URL, model IDs, structured-output behavior, usage fields, and any provider-specific parameters.
app/prompts.py	Holds evidence-grounding instructions, retrieval tools, the final JSON shape, and the prompt-injection boundary. It currently allows reported, synthesized, and inferred suggestions.	Built	Add explicit analogue-transfer, triangulation, assumptions, source/target differences, and validation-plan fields; tune with live evaluation.
app/tools.py	Validates and executes only approved retrieval tools. Enforces search count, top-k, neighbor-window, and total evidence-size limits.	Built	Verify behavior against live retrieval; add audit logging and clearer handling of zero-result searches.
app/agent.py	Runs the bounded tool-calling loop: ask model, validate tool call, execute retrieval, return evidence, and request a final structured answer.	Built	Test native Parley tool calls; add one controlled malformed-output repair attempt and better terminal handling.
app/validation.py	Parses final JSON; validates required fields, citation IDs, confidence/origin values, and flags numeric claims absent verbatim from cited evidence.	Built	Validate transparent analogue transfer and derived numbers without rejecting disclosed uncertainty; add unit conversions and claim-support checks.

3. User interface, evaluation, and support scripts

Component / script	Primary function	Progress	Remaining work
app/cli.py	Provides a command-line entry point for debugging one complete question without the UI. Supports deterministic demo mode.	Built	Run live after retrieval and Parley are connected; improve diagnostic output and exit codes.
app/ui.py	Provides the Streamlit form, optional cell conditions, result sections, citations, evidence display, validation status, and token counts. It selects demo or live adapters from configuration.	Built	Run against live services; expose transfer rationale and uncertainty fields after the output schema is expanded; decide local versus hosted access.
app/demo.py	Provides deterministic synthetic retrieval and LLM backends so orchestration can be tested before external systems are available.	Built	Retain strictly for development; prevent demo mode in production deployment.
evaluation/run_evaluation.py	Runs a fixed question set and records answers, validation state, agent rounds, tool calls, and tokens in JSONL.	Partial	Currently demo-only. Add live adapter selection, latency, retrieval metrics, provider/model, and observed credit usage.
evaluation/summarize_results.py	Aggregates evaluation runs into citation validity, unsupported-number, refusal, latency, token, and estimated-cost metrics.	Not built	Implement after the live evaluation result schema is finalized.
tests/test_agent.py	Tests the bounded orchestration path and input rejection with deterministic backends.	Built	Add tests for round limits, invalid tools, zero evidence, provider errors, and malformed final output.
tests/test_validation.py	Tests valid and fabricated citations against supplied evidence.	Built	Add numerical claims, missing fields, invalid confidence, analogue-transfer disclosure, and unsupported inference cases.
tests/test_retrieval_adapter.py	Checks that retrieved candidates preserve IDs, pages, section paths, DOI, vector rank, vector similarity, and reranker score when converted into agent evidence.	Built	Add mocked SQL tests and live integration checks against a known Neon fixture.

4. Existing pipeline scripts requiring integration work

Component / script	Primary function	Progress	Remaining work
retrieval/retrieval.py	Shared two-stage pipeline: normalized GTE query embedding, pre-retrieval section filtering, pgvector top-30 candidate search, GTE cross-encoder reranking, optional per-paper diversity, and structured output.	Built	Evaluate relevance thresholds, diversity defaults, hybrid retrieval, adaptive candidate counts, and missing-heading behavior on labeled questions.
embedding/embed-and-store.py	Existing ingestion and vector baseline for gte-modernbert-base embeddings in PostgreSQL/pgvector; now supports reusable local embedding caches and separate embed/store stages.	Existing	Verify cache and database model compatibility; later add safe sync-preview/sync-delete for obsolete rows.
chunk/create-chunks.py	Existing Docling HybridChunker pipeline with overlap, corrections, quarantine, metadata, and stable IDs.	Existing	Normalize section labels if needed. Optionally exclude introduction/abstract chunks during generation; query-time filtering is sufficient for the first live prototype.
parse_pdf/docling-converter.py	Existing PDF-to-Markdown and Docling JSON conversion with provenance, checksums, metadata, and optional image export.	Existing	No immediate application change required. Validate metadata consistency if citations need richer author/DOI fields.

5. Non-script infrastructure

Component / script	Primary function	Progress	Remaining work
Neon PostgreSQL/pgvector	Stores paper chunks, metadata, page numbers, model identifiers, and vectors. Performs cosine candidate search before reranking.	Partial	Database exists and schema mapping is implemented in the adapter, but it has not been verified with a live query.
MIT Parley API	Runs the hosted LLM and exposes an OpenAI-compatible endpoint with institutionally managed credits and privacy terms.	Not built	Obtain key privately; list models; select model; test chat, tools, JSON, quotas, and usage reporting.
Query embedding and reranking runtime	Lazily loads Alibaba-NLP/gte-modernbert-base for compatible query vectors and gte-reranker-modernbert-base for cross-encoder ranking; reuses both during the app session.	Built	First live run must download/cache models and verify memory, latency, 768 dimensions, and Streamlit session behavior.
Secrets/configuration	Automatically loads a repository-root .env file containing DATABASE_URL, LLM_API_KEY, LLM_MODEL, and limits. .gitignore excludes the real file; .env.example documents names.	Built	User must enter the three live values. A hosted deployment should use a secret manager rather than the local file.
Usage and budget controls	Limits LLM calls, context, searches, and output; records tokens and eventually estimates credits per question.	Partial	Token limits exist. Dollar/credit conversion, cumulative budget, remaining-balance display, and hard application cutoff are not built.
Deployment and access	Makes the Streamlit interface reachable by both collaborators while protecting credentials.	Not built	Initially run locally with individual keys, or choose a host, authentication, per-user secret strategy, and uptime plan.
Logging and observability	Records provider/model, retrieval decisions, validation results, latency, tokens, and failures without logging secrets.	Partial	Basic execution values exist in results/UI. Add structured logs, latency, request IDs, and sanitized error records.

6. Required order of operations

1. Review the pulled retrieval pipeline and the application adapter integration.
2. Enter DATABASE_URL, LLM_API_KEY, and an enabled Parley model ID in the Git-ignored .env file.
3. Directly verify semantic search, introduction/abstract exclusion, paper filters, metadata, and neighbor retrieval against Neon.
4. Verify the chosen model is enabled for the Parley key.
5. Test one Parley chat request, then JSON output, then native tool calling and usage fields.
6. Run one live CLI question end to end; fix adapter and provider mismatches.
7. Add one controlled output-repair attempt and complete failure/rate-limit handling.
8. Convert the evaluation runner from demo-only to selectable live backends.
9. Build summarize_results.py and run a curated 10-20 question evaluation set.
10. Evaluate retrieval thresholds, source diversity, neighbor expansion, and missing-heading behavior using labeled questions.
11. Expand prompts and validation for disclosed analogue transfer, triangulation, assumptions, source-target differences, and experimental validation needs.
12. Run the Streamlit UI against live services and verify citation transparency.
13. Add Parley credit estimation, cumulative limits, and safe quota-exhaustion behavior.
14. Decide local collaboration versus deployment; implement authentication/secrets if hosted.

15. Review all uncommitted files, update the root README, commit, and push.

7. Verification completed so far

- Five offline unit tests pass, including candidate-to-evidence provenance mapping.
- The local branch was fast-forwarded to the teammate's three retrieval and embedding-cache commits without overwriting uncommitted application work.
- The application imports and uses the actual GTE vector retrieval and cross-encoder reranking functions.
- The .env loader is installed and verified; the real .env file is ignored by Git.
- The deterministic CLI agent retrieves synthetic evidence and produces a citation-valid JSON answer.
- The three-question demo evaluation runner completes and writes JSONL results.
- The Streamlit server starts successfully in demo mode.
- No live Neon-to-Parley end-to-end test has been performed.
- No Parley credits have been consumed by this application work.

8. Definition of a usable first release

The first release is complete when a user can submit a question, the application retrieves filtered and reranked evidence from Neon, the Parley model may perform no more than the configured retrieval rounds, every protocol suggestion cites supplied chunks and discloses whether it is directly reported, adapted, triangulated, or hypothetical, invalid output is blocked or repaired once, usage is recorded, and the Streamlit interface clearly reports evidence, transfer assumptions, limitations, and safety boundaries. Direct charging-hardware control is explicitly out of scope.